

01-406-093-203

การพัฒนาเว็บแอปพลิเคชันด้วยเฟรมเวิร์ค

โดย นายศุภมิตร จันทริทวงศ์

- สาขาวิชาระบบสารสนเทศ คณะบริหารธุรกิจ

สัปดาห์ที่ 15

• การพัฒนา RESTFUL API และการยืนยันตัวตนด้วย LARAVEL SANCTUM

01-406-093-203 LARAVEL COURSE

เช็คชื่อเข้าชั้นเรียน

- ให้นักศึกษาสแกน QR Code นี้เพื่อเข้าสู่ระบบเช็คชื่อเข้าเรียน
- ระบบจะทำการบันทึกประวัติการเข้าเรียนในวิชานี้อัตโนมัติ

สแกนเพื่อเช็คชื่อเข้าชั้นเรียน (สัปดาห์ที่ 15)



สแกน QR Code เพื่อเช็คชื่อเข้าเรียนของสัปดาห์นี้

อาจารย์ผู้บรรยาย: ศุภมิตร จันทร์โทวงศ์

สไลด์ที่ 2 / 36

สัปดาห์ที่ 15 · การพัฒนา RESTFUL APIS และการยืนยันตัวตนด้วย LARAVEL SANCTUM

01-406-093-203 LARAVEL COURSE

แบบทดสอบก่อนเรียน (Pre-test)

1. RESTful API แตกต่างจากเว็บแอปพลิเคชันแบบเดิม (Web Page) อย่างไร?

- A. รับผิดชอบเฉพาะด้านเท่านั้น
- B. เน้นการรับส่งเฉพาะข้อมูลในรูปแบบ JSON หรือ XML ไม่ส่งโครงสร้างหน้าเว็บ HTML ไปด้วย เพื่อให้นัก Frontend หรือ Mobile ทำเป็นระบบแสดงผล
- C. ประมวลผลเร็วกว่าด้วยภาษา C++
- D. ไม่ต้องเชื่อมต่อกับฐานข้อมูล

3. หลังจากติดตั้ง Sanctum แล้ว Trait ใดที่ต้องนำไป use ไว้ในโมเดล User (app/Models/User.php) เพื่อให้นักพัฒนาสามารถออก Token ได้?

- A. HasApiTokens
- B. SanctumTokens
- C. Authenticatable
- D. CanIssueToken

5. เหตุใดในการทดสอบ API ด้วย Postman หรือโปรแกรม Client จึงควรกำหนด Header 'Accept: application/json' เสมอ?

- A. เพื่อเพิ่มความเร็วในการเชื่อมต่ออินเทอร์เน็ต
- B. เพื่อบังคับให้ Laravel ตอบกลับผลลัพธ์ (รวมถึงข้อผิดพลาดและ Validation Error) เป็น JSON เสมอ ป้องกันการ Redirect ในหน้า HTML
- C. เพื่อเปิดใช้งาน HTTPS อัตโนมัติ
- D. เพื่อให้ผู้ใช้สามารถอ่านภาษาไทยได้

7. คำสั่งใดที่ใช้สร้างคลาส API Resource เพื่อแปลงและจัดโครงสร้างข้อมูล Model ให้อยู่ในฟอร์แมต JSON ตามต้องการ?

- A. php artisan make:resource BlogResource
- B. php artisan maketransformer BlogTransformer
- C. php artisan make:json BlogJson
- D. php artisan make:response BlogResponse

9. คำสั่งใดที่ใช้ในการสร้างโทเค็นข้อความธรรมดา (Plain Text Token) ให้แก่ผู้ใช้งานหลังตรวจสอบการเข้าสู่ระบบสำเร็จ?

- A. \$user->createToken('token_name')->plainTextToken
- B. \$user->generateToken('token_name')
- C. Sanctum::createToken(\$user)

2. ใน Laravel 12 และ Laravel 13 คำสั่ง Artisan ใดสำหรับติดตั้งโครงสร้างระบบ API และ Laravel Sanctum แบบอัตโนมัติ?

- A. php artisan install:api
- B. composer require laravel/sanctum && php artisan vendor:publish
- C. php artisan make:auth --api
- D. php artisan sanctum:setup

4. ข้อใดคือการส่ง Token ยืนยันตัวตนผ่าน HTTP Request Headers ตามมาตรฐาน Bearer Token ที่ถูกต้อง?

- A. Authorization: Bearer {token}
- B. Token: Bearer {token}
- C. Access-Token: {token}
- D. Authentication: Token {token}

6. หากต้องการสร้าง Controller สำหรับจัดการข้อมูลแบบ RESTful API โดยไม่สร้าง Method แสดงหน้าเว็บ (create, edit) ควรใช้คำสั่งใด?

- A. php artisan make:controller Api/BlogController --api
- B. php artisan make:controller Api/BlogController --resource
- C. php artisan make:controller Api/BlogController --json
- D. php artisan make:controller Api/BlogController --simple

8. มีดัดแปลงตัวใดที่ใช้สำหรับป้องกัน Route ใน routes/api.php เพื่อให้เฉพาะผู้ที่ได้รับ Token ที่ถูกต้องเท่านั้นจึงจะเข้าถึงได้?

- A. auth:sanctum
- B. auth:api
- C. sanctum:guard
- D. token:check

10. เมื่อผู้ใช้ส่ง Logout และต้องการลบ Token ที่กำลังใช้งานอยู่ใน Request ปัจจุบันออกเพื่อง่ายต่อการดึง ควรใช้คำสั่งใด?

- A. \$request->user()->currentAccessToken()->delete();
- B. \$request->user()->tokens()->delete();
- C. Sanctum::logout(\$request);

Roadmap & ภาพรวมการพัฒนา RESTful API ด้วย Laravel Sanctum

- RESTful API เป็นหัวใจสำคัญในการทำ Backend ให้ระบบเชื่อมต่อกับ Frontend สมัยใหม่ (React, Vue, Next.js หรือ Mobile Apps)
- 1. พื้นฐาน RESTful API Concepts (HTTP Methods, Status Codes)
- 2. ติดตั้ง & คอมpile Laravel Sanctum (Token-based Authentication)
- 3. ระบบ Authentication (Register, Login, Logout, Tokens)
- 4. การสร้าง CRUD API ด้วย API Resource & Form Request
- 5. ป้องกัน Route ด้วย Sanctum Middleware (auth:sanctum)
- 6. Error Handling & Standard API Response Structure
- 7. การทดสอบด้วย Postman / Bruno / Thunder Client

อาจารย์ผู้บรรยาย: ศุภมิตร อัมกรโกวงศ์

สัปดาห์ 4 / 36

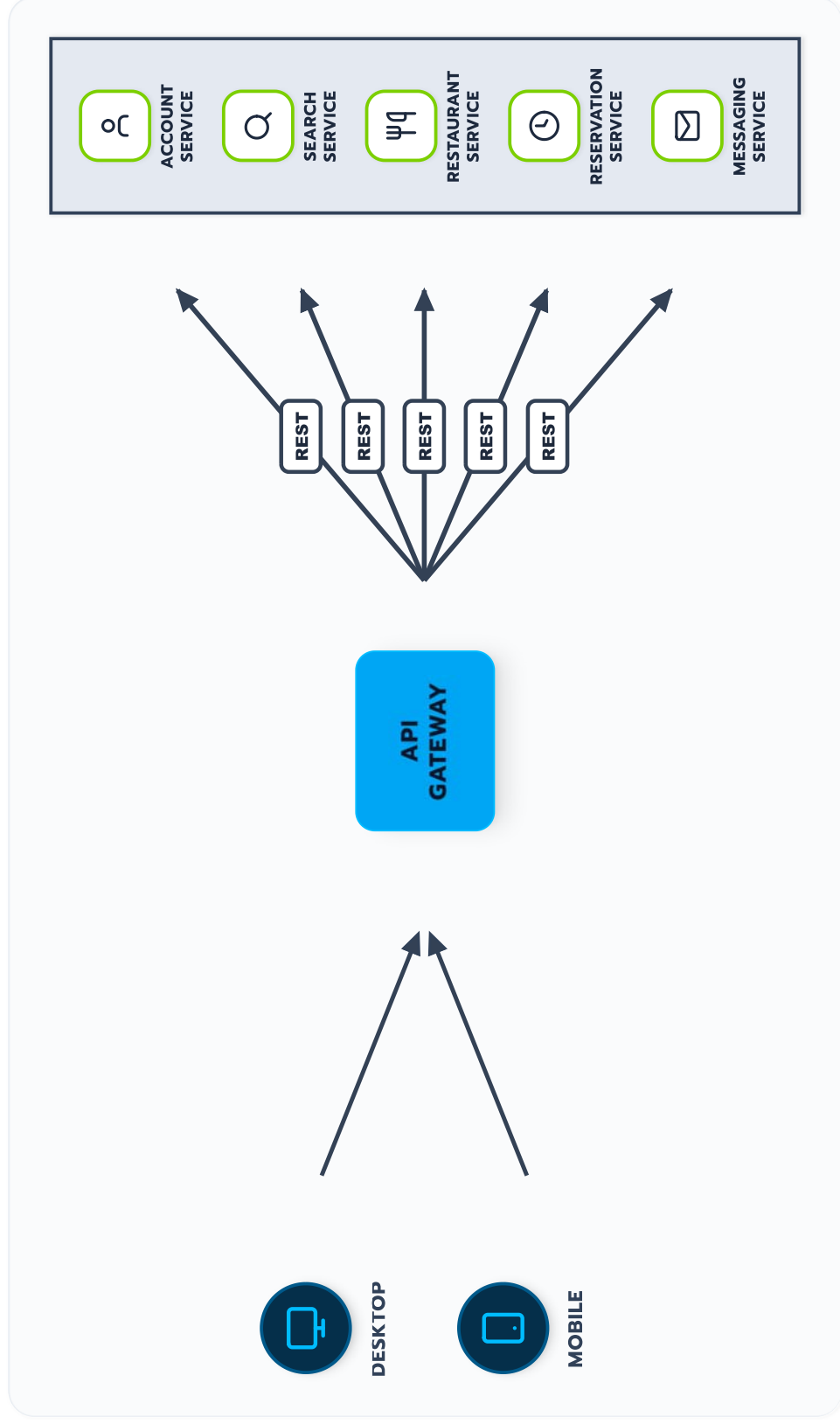
RESTful API & Sanctum Workflow Roadmap

1. **RESTful Concepts**
HTTP Methods (GET/POST/PUT/DELETE) และ Status Codes
2. **Install Sanctum**
php artisan install:api และ migrate ตาราง personal_access_tokens
3. **Auth API Controller**
Register, Login (ส่ง Plain Text Token), Logout
4. **CRUD & Resource**
BlogController และ BlogResource จัดรูปแบบ JSON
5. **Route Protection**
แยก Public Routes และ Protected Routes ด้วย middleware auth:sanctum
6. **Postman Testing**
กำหนด Header Accept: application/json และทดสอบ Bearer Token

RESTful API คืออะไร ?

- RESTful - RESTful Web Service (RWS) คือ Web Service ที่ใช้สถาปัตยกรรม REST
- RWS จะอนุญาตให้ระบบ Request และเข้าถึง Resource บนเว็บโดยใช้ชุดคำสั่งที่กำหนดเอาไว้ล่วงหน้า
- การโต้ตอบของระบบที่ใช้ REST จะอยู่บนพื้นฐานของ Hypertext Transfer Protocol (HTTP)
- Request จะส่งคำขอไปยัง URI ที่กำหนด และจะได้ response กลับมาเป็น Payload ในแบบ HTML, XML, JSON หรือ format อื่นๆ
- โดย RESTful จะประกอบไปด้วย:
 1. Client - ผู้ที่เข้ามาเป็น Request Resource
 2. Server - ผู้ให้บริการ Resource

RESTful API คืออะไร ?



JSON Format ?

- JSON (JavaScript Object Notation) คือรูปแบบมาตรฐานที่นิยมสูงในการรับส่งข้อมูลระหว่าง Client และ Server
- มีโครงสร้างเป็น Key-Value Pair คล้าย Array หรือ Object ทำให้อ่านเข้าใจง่ายและรองรับทุกภาษาโปรแกรม
- ตัวอย่างโครงสร้าง JSON ที่ได้จากการตอบกลับของ RESTful API:
- 필드 data: เก็บข้อมูลหลักที่ส่งกลับมา
- 필드 id: รหัสระบุรายการข้อมูล
- 필드 title: หัวข้อบทความ
- 필드 author: ข้อมูลผู้เขียนซึ่งเป็น Object ด้านใน (Nested Object)

code-example.json

```
{
  "data": {
    "id": 1,
    "title": "RESTful API with Laravel Framework",
    "description": "...",
    "author": {
      "id": 2580,
      "name": "Nirusduddin Mateh"
    }
  }
}
```

HTTP Method ที่ต้องรู้

- 1. Get - เป็นการร้องขอ / ดึงข้อมูลจาก Resource
- 2. Post - เป็นการสร้าง / เพิ่มข้อมูลใหม่ใน Resource
- 3. Put - เป็นการอัปเดต / แก้ไขข้อมูลที่มีอยู่แล้ว หรือสร้างใหม่ใน Resource
- 4. Delete - เป็นการลบข้อมูลที่มีอยู่แล้วใน Resource

code-example.php

```
// การนับคู่ HTTP Method กับการจัดการบทความ (Blogs)
// GET /api/blogs -> ดึงรายการบทความทั้งหมด
// GET /api/blogs/{id} -> ดึงข้อมูลบทความตาม ID
// POST /api/blogs -> สร้างบทความใหม่
// PUT /api/blogs/{id} -> แก้ไขบทความเดิม
// DELETE /api/blogs/{id} -> ลบบทความทิ้ง
```

HTTP Response Status Code ที่ต้องรู้

- 200 OK - ดำเนินการร้องขอข้อมูลสำเร็จลุล่วงด้วยดี
- 201 Created - สร้างข้อมูลใหม่ใน Resource สำเร็จเรียบร้อยแล้ว
- 401 Unauthorized - ยืนยันตัวตนไม่ผ่าน หรือไม่มี Token แบบมา
- 403 Forbidden - ไม่มีสิทธิ์เข้าถึงข้อมูลในส่วนนี้
- 404 Not Found - ไม่พบ Resource หรือ URI ที่ร้องขอ
- 422 Unprocessable Entity - ข้อมูลนำเข้าไม่ผ่านการตรวจสอบ (Validation Failed)
- 500 Internal Server Error - เกิดข้อผิดพลาดฝั่งเซิร์ฟเวอร์

HTTP Headers & หลักการ Stateless ที่ต้องรู้

- Stateless - RESTful API จะไม่จดจำ Session ของ Client ทุก Request ต้องส่งข้อมูลระบุตัวตนครบถ้วนเสมอ
- Accept: application/json - บอกเซิร์ฟเวอร์ว่า Client ต้องการผลลัพธ์เป็น JSON เสมอ (ป้องกันการ Redirect ไปหน้า HTML)
- Content-Type: application/json - แจ้งเซิร์ฟเวอร์ว่าข้อมูลใน Request Body ส่งมาในรูปแบบ JSON
- Authorization: Bearer token - แวน Token เพื่อใช้ยืนยันตัวตนสำหรับ Laravel Sanctum

code-example.http

```
// ตัวอย่าง HTTP Request Headers ในการเขียนใช้ RESTful API
POST /api/blogs HTTP/1.1
Host: 127.0.0.1:8000
Accept: application/json
Content-Type: application/json
Authorization: Bearer 1|a8f9c7e2b3d4...

{
  "title": "เรียนรู้ RESTful API",
  "content": "เนื้อหาบทความใหม่"
}
```

Step 1: ติดตั้งและเปิดใช้งาน API Scaffolding (install:api)

- รับคำสั่ง `php artisan install:api` ใน Terminal ของโปรเจกต์
- คำสั่งนี้จะสร้างไฟล์เส้นทาง `routes/api.php` ให้โดยอัตโนมัติ
- ทำการติดตั้งและตั้งค่าแพ็คเกจ `laravel/sanctum` พร้อมไฟล์คอนฟิก `config/sanctum.php`
- สร้าง Migration สำหรับตาราง `personal_access_tokens` เพื่อใช้บันทึกโทเค็นของผู้ใช้

code-example.bash

```
# ติดตั้ง API Scaffolding และ Sanctum
php artisan install:api

# ผลลัพธ์ที่ระบบดำเนินการ:
# - One new database migration has been published.
# - The [routes/api.php] file has been published.
# - Laravel Sanctum installed successfully.
```

Step 2: ตรวจสอบ Trait HasApiTokens ใน User Model (app/Models/User.php)

- เปิดไฟล์ `app/Models/User.php`
- นำเข้า Trait ด้วยคำสั่ง `use Laravel\Sanctum\HasApiTokens;`
- เรียกใช้ `use HasApiTokens, Notifiable;` ภายในคลาส `User`
- Trait นี้จะเพิ่มเมธอดสำคัญ เช่น `$user->createToken()` และ `$user->tokens()` สำหรับจัดการ Token

code-example.php

```
// app/Models/User.php
namespace App\Models;

use Laravel\Sanctum\HasApiTokens;
use Illuminate\Notifications\Notifiable;
use Illuminate\Foundation\Auth\User as Authenticatable;

class User extends Authenticatable
{
    use HasApiTokens, Notifiable;

    protected $fillable = [
        'name',
        'email',
        'password',
    ];
};
```

Step 3: รับ Migration เพื่อสร้างตาราง personal_access_tokens

- รับคำสั่ง `php artisan migrate` ใน Terminal
- Laravel จะสร้างตาราง `personal_access_tokens` ในฐานข้อมูล MySQL
- ตารางนี้มีคอลัมน์สำคัญ: `tokenable_type`, `tokenable_id`, `name`, `token` (Hash SHA-256) และ `abilities`
- สามารถตรวจสอบตารางที่ถูกสร้างขึ้นได้ผ่านโปรแกรม phpMyAdmin หรือ Database GUI

code-example.bash

```
# สั่งรับ Migration สร้างตาราง Tokens
php artisan migrate

# โค้ดสร้างตาราง personal_access_tokens:
# - id (bigint unsigned)
# - tokenable_type (App\Models\User)
# - tokenable_id (User ID)
# - name (เช่น myapptoken)
# - token (SHA-256 hash ของโทเค้น)
# - abilities (สิทธิ์การเข้าถึง เช่น [ 'STRING 0' ])
# - last_used_at (เวลาที่ใช้ล่าสุด)
# - created_at & updated_at
```

Step 4: สร้าง Auth API Controller (Api/AuthController)

- สร้าง Controller ไว้ไฟล์เดียว **Api/** เพื่อแยกออกจาก Web Controller หนึ่ง
- รับคำสั่ง `php artisan make:controller Api/AuthController`
- นำเข้า Class: **User** , **Hash** , **Request** และ **Response**
- เตรียมเมธอดหลัก 3 เมธอด: `register()` , `login()` และ `logout()`

```
code-example.bash
# สร้าง Controller ไว้ไฟล์เดียว Api/
php artisan make:controller Api/AuthController

# ไฟล์จะถูกสร้างขึ้นที่:
# app/Http/Controllers/Api/AuthController.php
```

Step 5: เขียนฟังก์ชัน Register สมาชิกและส่งคืน Token

- เปิดไฟล์ `app/Http/Controllers/Api/AuthController.php`
- ตรวจสอบความถูกต้องของข้อมูลด้วย `$request->validate()`
- เข้ารหัสผ่านด้วย `Hash::make($fields['password'])`
- สร้างผู้ใช้ใหม่ฐานข้อมูลและสร้าง Token ด้วย `$user->createToken('myapptoken')->plainTextToken`
- ส่ง Response เป็น JSON กลับไปพร้อม HTTP Status Code 201 (Created)

code-example.php

```
// app/Http/Controllers/Api/AuthController.php
namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;

class AuthController extends Controller
{
    public function register(Request $request)
    {
        $fields = $request->validate([
            'name' => 'required|string|max:255',
            'email' => 'required|string|email|unique:users,email',
            'password' => 'required|string|confirmed|min:6'
        ]);
```

Step 5.1: เชื่อม Route ใน routes/api.php สำหรับ Register

- เปิดไฟล์ `routes/api.php`
- นำเข้า Controller ด้านบน: `use App\Http\Controllers\Api\AuthController;`
- เพิ่มคำสั่งกำหนด Route:
`Route::post('/register', [AuthController::class, 'register']);`
- เส้นทางนี้เปิดเป็น Public Route เพื่อให้ผู้ใช้งานภายนอกสามารถส่งคำขอสมัครสมาชิกได้โดยไม่ต้องแบบ Token

code-example.php

```
<?php

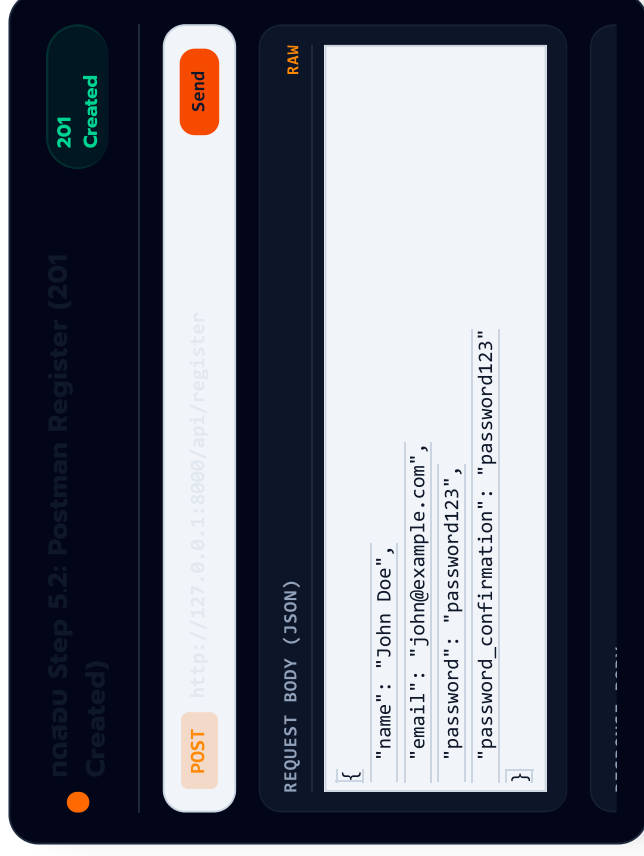
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\AuthController;

Route::get('/user', function (Request $request) {
    return $request->user();
})->middleware('auth:sanctum');

Route::post('/register', [AuthController::class, 'register']);
```

Step 5.2: ทดสอบ Register API ใน Postman

- เปิดโปรแกรม Postman สร้าง Request ใหม่ เลือก Method เป็น **POST**
- กำหนด URL: `http://127.0.0.1:8000/api/register` (หรือตามพอร์ตที่รับ)
- ตั้งค่า Headers สำคัญ: **Accept: application/json** และ **Content-Type: application/json**
- เลือก Body -> **raw** -> **JSON** และส่งข้อมูล: name, email, password, password_confirmation
- กด ****Send**** แล้วตรวจสอบว่าได้ผลลัพธ์ ****Status 201 Created**** พร้อม Plain Text Token ใหม่



Step 6: เขียนฟังก์ชัน Login ตรวจสอบสิทธิ์และออก Token

- รับค่า email และ password จากคำขอ
- ค้นหาผู้ใช้จาก Email ในตาราง users ด้วย `User::where('email', ...)->first()`
- ตรวจสอบรหัสผ่านด้วย `Hash::check($fields['password'], $user->password)`
- หากไม่ถูกต้อง ส่ง Response 401 Unauthorized กลับทันที
- หากถูกต้อง สร้าง Plain Text Token ใหม่และส่งกลับเป็น JSON Status 200 OK

code-example.php

```
// app/Http/Controllers/AuthController.php
public function login(Request $request)
{
    $fields = $request->validate([
        'email' => 'required|string|email',
        'password' => 'required|string'
    ]);

    $user = User::where('email', $fields['email'])->first();

    // ตรวจสอบว่า User และ Password ตรงกันหรือไม่
    if (!$user || !Hash::check($fields['password'], $user->password)) {
        return response()->json([
            'status' => 'error',
            'message' => 'อีเมลหรือรหัสผ่านไม่ถูกต้อง'
        ], 401);
    }
}
```

Step 6.1: เชื่อม Route ใน routes/api.php สำหรับ Login

- เปิดไฟล์ routes/api.php
- เพิ่ม Route: Route::post('/login', [AuthController::class, 'login']);
- จัดเป็น Public Route เช่นเดียวกับ /register เพื่อให้ใครเอนต์ส่งคำขอยืนยันตัวตนได้ก่อนได้รับ Token
- เมื่อส่งคำขอถูกต้อง ระบบจะสร้างและตอบกลับ Plain Text Token สำหรับใช้ใน Protected Routes ต่อไป

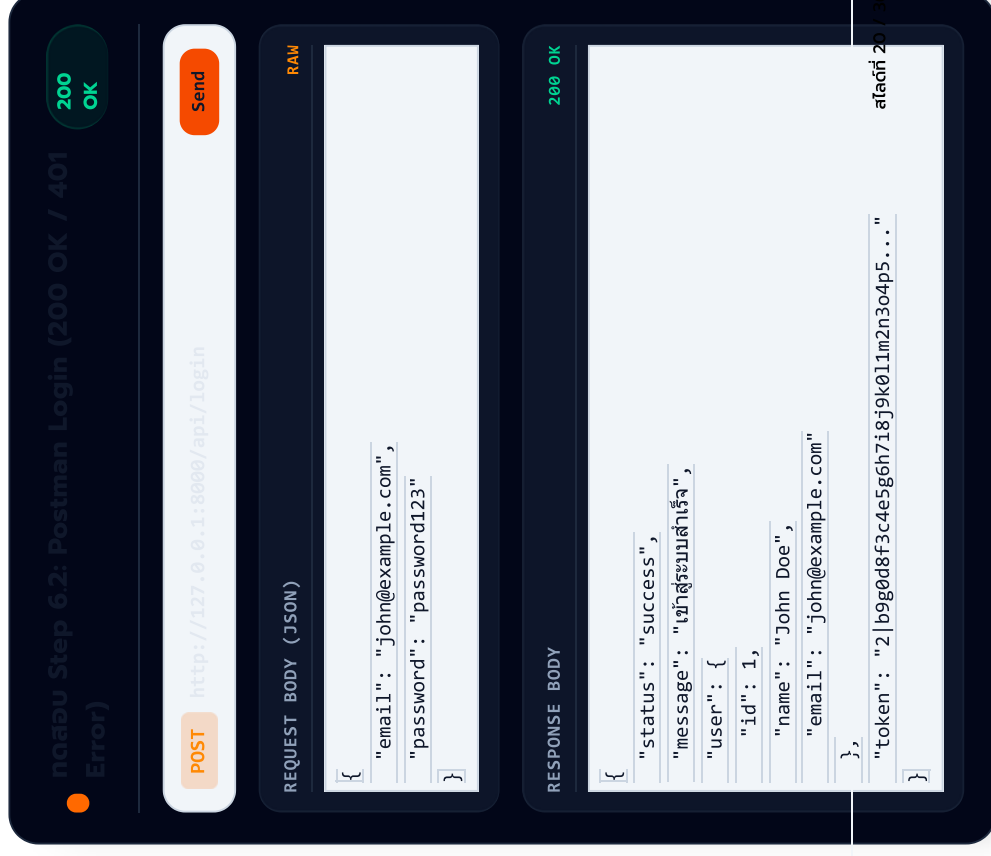
code-example.php

```
<?php  
  
use Illuminate\Http\Request;  
use Illuminate\Support\Facades\Route;  
use App\Http\Controllers\Api\AuthController;  
  
Route::get('/user', function (Request $request) {  
    return $request->user();  
})->middleware('auth:sanctum');  
  
Route::post('/register', [AuthController::class, 'register']);  
Route::post('/login', [AuthController::class, 'login']);
```

Step 6.2: ทดสอบ Login API ใน Postman

- เปิด Postman สร้าง Request ใหม่: `POST http://127.0.0.1:8000/api/login`
- ตั้งค่า Headers: `Accept: application/json` และ `Content-Type: application/json`
- เลือก Body -> `raw` -> `JSON` ส่งข้อมูล email และ password ที่เคยลงทะเบียนไว้
- กรณีรหัสผ่านถูกต้อง: จะได้ Status `**200 OK**` พร้อมค่า `token` ตัวใหม่สำหรับใช้งาน
- กรณีรหัสผ่านผิด (Error Check): จะได้ Status `**401 Unauthorized**` พร้อมข้อความแจ้งเตือนทันที

อาจารย์ผู้บรรยาย: ศุภมิตร จันทร์โทวงศ์



Step 7: เขียนฟังก์ชัน Logout ยกเลิกและลบ Token

- ผู้ใช้ต้องแนบ Bearer Token เข้ามาในคำขอผ่าน Middleware `auth:sanctum`
- ใช้ `$request->user()->currentAccessToken()->delete()` เพื่อลบเฉพาะ Token ที่กำลังใช้งานปัจจุบัน
- กรณีต้องการบังคับให้ออกจากระบบทุกเครื่อง (Revoke All) ให้ใช้ `$request->user()->tokens()->delete()`
- ส่งข้อความยืนยันการออกจากระบบกลับเป็น JSON

code-example.php

```
// app/Http/Controllers/Api/AuthController.php
public function logout(Request $request)
{
    // ลบเฉพาะ Token ปัจจุบันที่ใช้เรียก API นี้
    $request->user()->currentAccessToken()->delete();

    return response()->json([
        'status' => 'success',
        'message' => 'ออกจากระบบเรียบร้อยแล้ว (Token ถูกยกเลิก)'
    ], 200);
}
```

Step 7.1: จัดกลุ่ม Protected Routes ด้วย auth:sanctum ใน routes/api.php

- เปิดไฟล์ `routes/api.php`
- คอลเลกชันทางที่ต้องการความปลอดภัยด้วย `Route::middleware('auth:sanctum')->group(...)`
- ย้าย Route `/user` และเพิ่ม `Route::post('/logout', [AuthController::class, 'logout']);` ไว้ในกลุ่มนี้
- ทุกค่าขอที่เรียกมายัง Route ในกลุ่มนี้จะต้องแนบ Header `Authorization: Bearer <token>` เสมอ

code-example.php

```
<?php
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\AuthController;

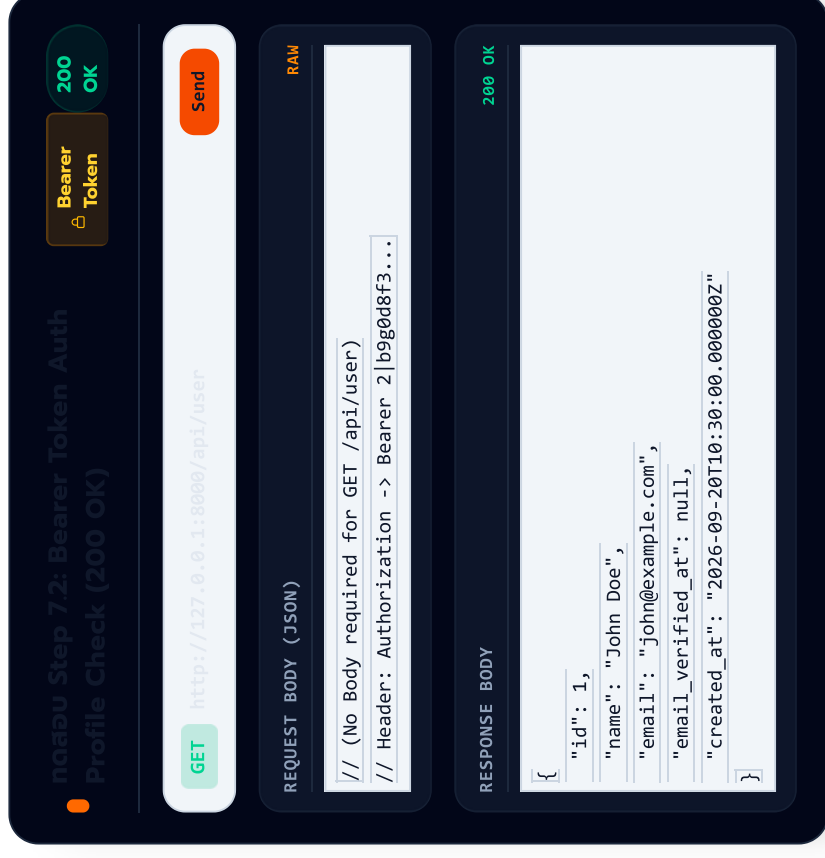
Route::middleware('auth:sanctum')->group(function () {
    Route::get('/user', function (Request $request) {
        return $request->user();
    });
    Route::post('/logout', [AuthController::class, 'logout']);
});

Route::post('/register', [AuthController::class, 'register']);
Route::post('/login', [AuthController::class, 'login']);
```

Step 7.2: ทดสอบ Protected Route ใน Postman

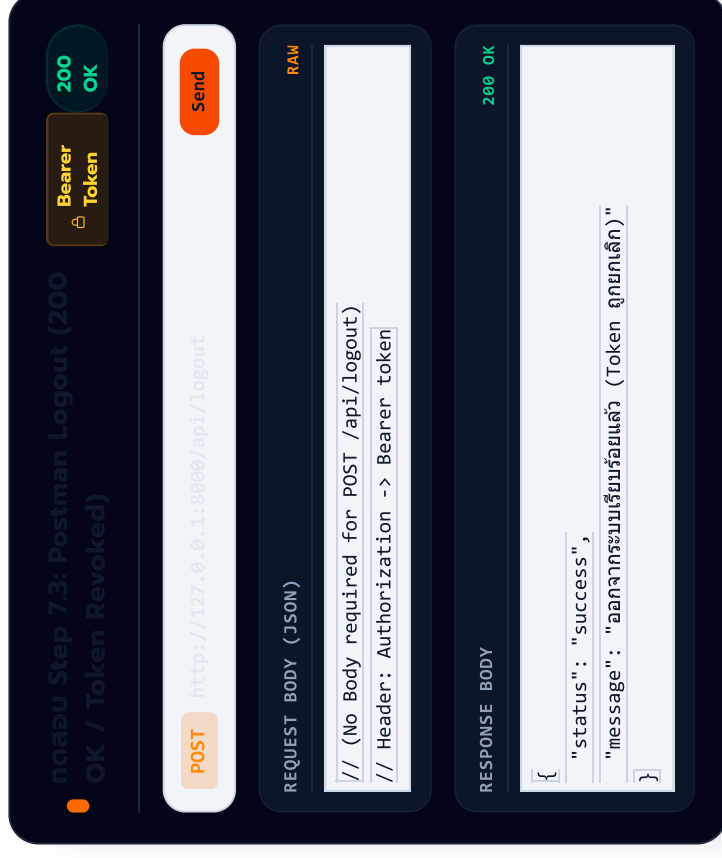
(GET /api/user)

- สร้าง Request: GET `http://127.0.0.1:8000/api/user`
- ไปที่แท็บ **Authorization** เลือก Type เป็น **Bearer Token**
- วาง Token ที่ได้จากตอน Login ลงในช่อง Token แล้วกด **Send**
- ตรวจสอบว่าได้ผลลัพธ์ **Status 200 OK** พร้อมข้อมูลโปรไฟล์ของผู้ใช้งาน (id, name, email)
- หากไม่แนบ Token หรือ Token ไม่ถูกต้อง ระบบจะตอบกลับ **401 Unauthorized** นั่น



Step 7.3: ทดสอบ Logout ยกเลิก Token ใน Postman (POST /api/logout)

- สร้าง Request: POST `http://127.0.0.1:8000/api/logout`
- ไปที่แท็บ ****Authorization**** เลือก Type เป็น ****Bearer Token**** และแบบ Token ปัจจุบัน
- ในส่วน Body เลือก **raw** -> **JSON** (ไม่ต้องส่งฟิลด์ข้อมูลใดๆ)
- กด ****Send**** ตรวจสอบว่าได้ผลลัพธ์ ****Status 200 OK**** พร้อมข้อความ **ออกจากระบบเรียบร้อยแล้ว (Token ถูกยกเลิก)**
- ทดสอบยังซ้ำด้วย Token เดิม: จะติด ****401 Unauthorized**** กันที่ ยืนยันว่า Token ถูกลบออกจากฐานข้อมูลจริง



Step 8: สร้าง API Resource (BlogResource) สำหรับจัดระเบียบ JSON

- สร้างคลาส Resource ด้วยคำสั่ง `php artisan make:resource BlogResource`
- ไฟล์จะถูกสร้างขึ้นที่ `app/Http/Resources/BlogResource.php`
- API Resource ทำหน้าที่เป็นตัวแปลงข้อมูล (Transformer Layer) ระหว่าง Eloquent Model และ JSON Response
- ช่วยควบคุมฟิลด์ที่ต้องการแสดงหรือซ่อน และจัดฟอร์แมตวันที่เวลาให้ตรงมาตรฐาน

```
code-example.bash

# สร้าง API Resource
php artisan make:resource BlogResource

# ไฟล์จะอยู่ที่:
# app/Http/Resources/BlogResource.php
```

Step 9: กำหนดรูปแบบ JSON ใน BlogResource

- เปิดไฟล์ `app/Http/Resources/BlogResource.php`
- กำหนดคีย์ในแบบออก `toArray(Request $request): array`
- เลือกส่งเฉพาะฟิลด์ที่จำเป็น: `id`, `title`, `content`, `status`
- จัดรูปแบบวันที่ด้วย `$this->created_at->format('Y-m-d H:i:s')`

code-example.php

```
// app/Http/Resources/BlogResource.php
namespace App\Http\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class BlogResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'title' => $this->title,
            'content' => $this->content,
            'status' => (bool) $this->status,
            'created_at' => $this->created_at ? $this->created_at->format('Y-m-d H:i:s')
```

Step 10: สร้าง BlogController แบบ API (--api)

- รับคำสั่ง `php artisan make:controller Api/BlogController --api`
- ตัวเลือก `--api` จะสร้างเฉพาะแอดสำหรับ RESTful API (index, store, show, update, destroy)
- จะตัดเมธอด `create()` และ `edit()` ที่ใช้สำหรับแสดงหน้าฟอร์ม HTML ออกให้อัตโนมัติ
- นำเข้า Model `Blog` และ `BlogResource` ไว้ด้านบนของ Controller

code-example.bash

```
# สร้าง API Resource Controller
php artisan make:controller Api/BlogController --api

# เมธอดที่ได้:
# - index() : GET /api/blogs
# - store() : POST /api/blogs
# - show() : GET /api/blogs/{id}
# - update() : PUT/PATCH /api/blogs/{id}
# - destroy() : DELETE /api/blogs/{id}
```

Step 11: เขียนฟังก์ชัน CRUD ใน BlogController (index & store)

- เปิดไฟล์ `app/Http/Controllers/Api/BlogController.php`
- เมธอด `index()` : ดึงข้อมูลบทความแบบเรียงลำดับและแบ่งหน้าผ่าน `BlogResource::collection()`
- เมธอด `store()` : ตรวจสอบข้อมูล บันทึกบทความใหม่ และส่งกลับเป็น `new BlogResource($blog)` พร้อมสถานะ: 201

code-example.php

```
// app/Http/Controllers/Api/BlogController.php
namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\Blog;
use App\Http\Resources\BlogResource;
use Illuminate\Http\Request;

class BlogController extends Controller
{
    // GET /api/blogs
    public function index()
    {
        $blogs = Blog::latest()->paginate(10);
        return BlogResource::collection($blogs);
    }
}
```

Step 12: เขียนฟังก์ชัน CRUD ใน BlogController (show, update, destroy)

- เมธอด `show($id)` : ค้นหาความตาม ID หากพบส่ง `new BlogResource($blog)`
- เมธอด `update(Request $request, $id)` : ตรวจสอบความถูกต้อง อัปเดตข้อมูล และส่ง Resource พลัฟร์กลับ
- เมธอด `destroy($id)` : ค้นหาและสั่ง `$blog->delete()` พร้อมส่งข้อความยืนยันการลบกลับเป็น JSON

code-example.php

```
// app/Http/Controllers/Api/BlogController.php
// GET /api/blogs/{id}
public function show($id)
{
    $blog = Blog::findOrFail($id);
    return new BlogResource($blog);
}

// PUT /api/blogs/{id}
public function update(Request $request, $id)
{
    $blog = Blog::findOrFail($id);

    $request->validate([
        'title' => 'sometimes|required|string|max:255',
        'content' => 'sometimes|required|string'
```

Step 13: กำหนดเส้นทาง API ทั้งหมดใน routes/api.php

- เปิดไฟล์ routes/api.php ตรวจสอบการลงทะเบียนเส้นทาง
- Public Routes: /register , /login , GET /blogs , GET /blogs/{id} (เรียกได้โดยไม่ต้องใช้ Token)
- Protected Routes (auth:sanctum): /user , /logout , POST /blogs , PUT /blogs/{id} , DELETE /blogs/{id}
- ทุกค่าต่อไปยัง Protected Routes มุ่งกับแบบ Header Authorization: Bearer <token>

code-example.php

```
// routes/api.php
use App\Http\Controllers\Api\AuthController;
use App\Http\Controllers\Api\BlogController;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;

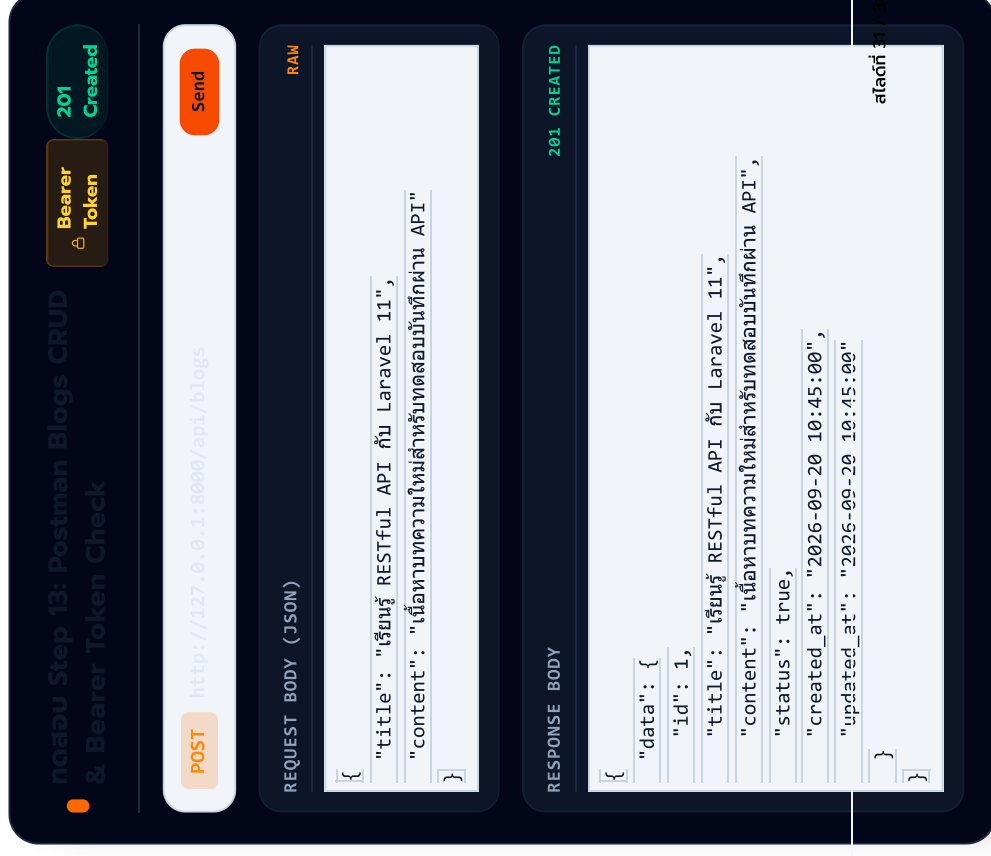
// 1. Public Routes
Route::post('/register', [AuthController::class, 'register']);
Route::post('/login', [AuthController::class, 'login']);
Route::get('/blogs', [BlogController::class, 'index']);
Route::get('/blogs/{id}', [BlogController::class, 'show']);

// 2. Protected Routes (ต้องแนบ Bearer Token)
Route::middleware('auth:sanctum')->group(function () {
    Route::get('/user', function (Request $request) {
        return $request->user();
    });
});
```

Step 13.1: ทดสอบ CRUD Blogs ใน Postman (GET /blogs & POST /blogs)

- ทดสอบ ****GET /api/blogs****: เลือก Auth เป็น No Auth กด Send เพื่อผลลัพธ์ JSON และ Pagination จาก BlogResource
- ทดสอบ ****POST /api/blogs****: แบบ Bearer Token และส่ง Body JSON (title, content) เพื่อบันทึกบทความใหม่
- ตรวจสอบสถานะผลลัพธ์ ****201 Created**** พร้อมโครงสร้างไฟล์วันที่ແຈງຜ່ານ Resource
- ทดสอบ ****GET /api/blogs/1**** และ ****DELETE /api/blogs/1**** เพื่อยืนยันการทำงานของ CRUD ครบวงจร

อาจารย์ผู้บรรยาย: ศุภมิตร จันทร์ทิววงศ์



Best Practices & ข้อควรระวังในการพัฒนา Laravel RESTful API

- Header Accept: application/json: ปัญหาของอดีตของผู้เริ่มต้นคือเวลา Validate ไม่ผ่าน Laravel จะพยายาม Redirect กลับหน้าเดิม (302) ถ้าไม่ใส่ Header นี้
- Token Abilities/Scopes: กำหนดสิทธิ์ของ Token ได้ เช่น \$user->createToken('token', ['read', 'write']) เพื่อจำกัดความสามารถของแอปพลิเคชันหลายๆทาง
- SPA vs Mobile API: Sanctum รองรับ 2 โหมดคือ: (1) Cookie-based / Session (สำหรับ Vue/React SPA โดเมนเดียวกัน) และ (2) Token-based (สำหรับ Mobile App หรือ Cross-domain)
- การเก็บรักษาการรหัสผ่าน: รหัสผ่านต้องเข้ารหัสด้วย Hash::make() เสมอ ห้ามบันทึกเป็น Plain Text
- Standard Response Structure: คืนค่า JSON โครงสร้างสม่ำเสมอ เช่น status, message, data เพื่อให้ฝั่ง Frontend จัดการได้ง่าย

สัปดาห์ที่ 15 · การพัฒนา RESTFUL APIS และการยืนยันตัวตนด้วย LARAVEL SANCTUM

01-406-093-203 LARAVEL COURSE

แบบทดสอบหลังเรียน (Post-test)

1. คำสั่งใดใช้สร้าง Controller สำหรับ RESTful API ที่มีเฉพาะเมธอด index, store, show, update, destroy โดยไม่สร้างหน้าเว็บฟอร์ม?

- A. php artisan make:controller Api/BlogController --api
- B. php artisan make:controller Api/BlogController --resource
- C. php artisan make:controller Api/BlogController --json
- D. php artisan make:controller Api/BlogController --service

3. การป้องกัน Protected Routes ในไฟล์ routes/api.php ต้องกำหนดเมธอดใดไว้ในกลุ่มเส้นทาง?

- A. Route::middleware('auth:sanctum')->group(function () { ... });
- B. Route::middleware('sanctum')->group(function () { ... });
- C. Route::middleware('auth:api')->group(function () { ... });
- D. Route::middleware('apitoken')->group(function () { ... });

5. เมื่อ Client ส่งคำขอสร้างข้อมูลใหม่สำเร็จ HTTP Status Code ใดที่เซิร์ฟเวอร์ควรส่งตอบกลับตามมาตรฐาน RESTful API?

- A. 201 Created
- B. 200 OK
- C. 204 No Content
- D. 301 Moved Permanently

7. หากข้อมูลนำเข้าไม่ผ่านการตรวจสอบความถูกต้องจาก Form Request (Validation Failed) เซิร์ฟเวอร์จะตอบกลับด้วยรหัสข้อผิดพลาดใด?

- A. 422 Unprocessable Entity
- B. 400 Bad Request
- C. 401 Unauthorized
- D. 500 Internal Server Error

9. หากผู้ใช้ต้องการ Logout จากเครื่องที่กำลังใช้งานอยู่ โดยไม่กระทบกับอุปกรณ์อื่นที่กำลังล็อกอินค้างไว้ ควรใช้คำสั่งใด?

- A. \$request->user()->currentAccessToken()->delete();
- B. \$request->user()->tokens()->delete();
- C. Auth::logout();
- D. \$request->user()->delete();

2. เมื่อสร้าง API ด้วย Laravel Sanctum ข้อใดคือคำสั่งในการสร้างและรับค่า Plain Text Token ที่ส่งกลับให้แอปพลิเคชันฝั่งไคลเอนต์เพียงครั้งเดียว?

- A. \$token = \$user->createToken('auth_token')->plainTextToken();
- B. \$token = \$user->generateToken('auth_token');
- C. \$token = Sanctum::issueToken(\$user);
- D. \$token = \$user->tokens()->makePlainText();

4. ประโยชน์หลักของการใช้งาน API Resource (เช่น BlogResource) ใน Laravel คือข้อใด?

- A. ช่วยกำหนดโครงสร้างและกรองข้อมูล JSON ที่ส่งกลับไปยังไคลเอนต์ได้อย่างปลอดภัยและเป็นมาตรฐาน
- B. ช่วยเร่งความเร็วในการเชื่อมต่อฐานข้อมูล
- C. ใช้สำหรับบันทึกข้อมูลลงฐานข้อมูลโดยตรง
- D. ใช้สำหรับสร้างหน้าเว็บ HTML อัตโนมัติ

6. หากผู้ใช้งานแบบ Token ที่ไม่ถูกต้อง หรือไม่ได้นับ Token มาใน Protected Route เซิร์ฟเวอร์จะตอบกลับด้วย Status Code ใด?

- A. 401 Unauthorized
- B. 403 Forbidden
- C. 404 Not Found
- D. 422 Unprocessable Entity

8. ในการทดสอบ Protected API ผ่านโปรแกรม Postman ค่า Token ต้องถูกนำไปตั้งค่าที่ส่วนใด?

- A. แท็บ Authorization เลือก Type เป็น 'Bearer Token' แล้ววาง Token
- B. แท็บ Params ใส่ ?token=..
- C. แท็บ Body แบบเป็นฟอร์มข้อมูล
- D. แท็บ Cookies ของเบราว์เซอร์

10. ข้อใดคือการส่ง Header ที่ถูกต้องเพื่อแจ้งให้เซิร์ฟเวอร์ทราบว่า Client ส่งข้อมูลเนื้อหา Request Body มาในรูปแบบ JSON?

- A. Content-Type: application/json
- B. Accept: application/json
- C. Data-Type: json
- D. Format: json

คำถามท้ายบทเรียน

เหตุใดการระบุ Header 'Accept: application/json' ในโปรแกรมทดสอบ API (เช่น Postman) จึงมีความสำคัญอย่างยิ่งสำหรับ Laravel API?

A. เพื่อให้เซิร์ฟเวอร์รับโค้ดเร็วขึ้น 2 เท่า

B. เพื่อให้ Laravel ส่งผลลัพธ์ผิดพลาด (เช่น Validation Error) กลับเป็น JSON แทนที่จะทำ HTTP 302 Redirect ไปหน้าเดิม

C. เพื่อเปิดการใช้งาน HTTPS อัตโนมัติ

D. เพื่อไม่ต้องกรอกรหัสผ่านในการล็อกอิน

คำอธิบาย: หากไม่มี Header 'Accept: application/json' เมื่อเกิด Validation Error ระบบ Laravel จะมองว่าเป็นคำขอจาก Web Form ธรรมดาและพยายาม Redirect กลับ (HTTP 302) แทนที่จะตอบกลับเป็น 422 JSON

สัปดาห์ที่ 15 · การพัฒนา RESTFUL APIS และการยืนยันตัวตนด้วย LARAVEL SANCTUM

01-406-093-203 LARAVEL COURSE

Workshop 15: พัฒนา RESTful Blog API & ระบบยืนยันตัวตนด้วย Laravel Sanctum

- 1 Step 1 ติดตั้ง API Scaffolding: รันคำสั่ง `php artisan install:api` เพื่อสร้าง routes/api.php และ config/sanctum.php
- 2 Step 2 ตั้งค่า Model User: เปิด `app/Models/User.php` นำเข้าและเรียกใช้ Trait `HasApiTokens` ภายในคลาส
- 3 Step 3 รัน Migration: สั่งรัน `php artisan migrate` เพื่อสร้างตาราง `personal_access_tokens` ในฐานข้อมูล
- 4 Step 4 สร้าง AuthController: รัน `php artisan make:controller Api/AuthController` สำหรับจัดการระบบสมาชิก
- 5 Step 5 พัฒนาเมธอด Register: ตรวจสอบ Validation และสร้าง Token ด้วย `$user->createToken('myapptoken')->plainTextToken`
- 6 Step 5.1 & 5.2 เชื่อม Route และทดสอบ Register ใน Postman: ยัง `POST /api/register` พร้อมตรวจสอบสถานะ: 201 Created
- 7 Step 6 พัฒนาเมธอด Login: ตรวจสอบ Hash::check() และส่งคืน Token ใหม่เป็น JSON Status 200 OK
- 8 Step 6.1 & 6.2 เชื่อม Route และทดสอบ Login ใน Postman: ยัง `POST /api/login` ตรวจสอบ Token และทดสอบ Error Check (401)
- 9 Step 7 พัฒนาเมธอด Logout: ใช้ `$request->user()->currentAccessToken()->delete()` เพื่อลบและยกเลิก Token
- 10 Step 7.1 - 7.3 ป้องกัน Route ด้วย Sanctum และทดสอบ Logout: จัดกลุ่ม `Route::middleware('auth:sanctum')` สำหรับ `/user` และ `/logout`
- 11 Step 8 & 9 สร้าง BlogResource: รัน `php artisan make:resource BlogResource` และจัดรูปแบบฟิลด์ตัวเวลาที่ให้เป็นมาตรฐาน

command_prompt.sh

```
// 1. ติดตั้ง API Scaffolding & รัน Migration
php artisan install:api
php artisan migrate

// 2. สร้าง Controllers & Resource
php artisan make:controller Api/AuthController
php artisan make:controller Api/BlogController --api
php artisan make:resource BlogResource

// 3. รัน Server สำหรับให้บริการ API บน Port 8000
php artisan serve
```

ใบงานที่ 15: พัฒนาระบบ RESTful Blog API พร้อมระบบยืนยันตัวตนด้วย Laravel Sanctum

ให้นักศึกษาพัฒนาชุด Web Service แบบ RESTful API สำหรับระบบบทความ (Blogs) ตามสถาปัตยกรรม Stateless โดยมีระบบจัดการผู้ใช้งาน (Register, Login, Logout) ควบคุมการออกและเอา Token ผ่าน Laravel Sanctum และระบบจัดการบทความ (CRUD) ที่แบ่งแยกระหว่าง Public Routes (อ่านข้อมูล) และ Protected Routes (เขียน แก้ไข ลบข้อมูลด้วย Bearer Token) พร้อมจัดทำเอกสารผลการทดสอบผ่าน Postman ครบทุกฟังก์ชัน

เกณฑ์การตรวจประเมินผล:

- การติดตั้ง & ตั้งค่าระบบ API: ติดตั้ง install:api, ตาราง personal_access_tokens ใน User Model **20%**
- ระบบ Authentication API: พัฒนาฟังก์ชัน Register **201 Created, Login**
- การพัฒนา CRUD API & BlogResource: สร้าง BlogController **--api, BlogResource จัดรูปแบบ JSON พาด id, title, content, status, วันที่**
- การควบคุมสิทธิ์ Route Protection: กำหนดกลุ่ม middleware('auth:sanctum') ใน routes/api.php สำหรับ /user, /logout และ POST/PUT/DELETE blogs **15%**
- รายงานผลการทดสอบด้วย Postman: แบบ Header Accept: application/json, Content-Type, Bearer Token และจับภาพผลลัพธ์ทุกสถานะ ทั้ง Success และ Error Check 401/422 **10%**

